

Real-Time Maritime Surveillance — 24-Hour Hackathon Build Blueprint

1. Problem Statement, Decoded

What the judges actually want to see: a backend that can chew through **50,000+ fake ship-position messages every second**, decide **in milliseconds** whether each one is inside a forbidden zone, and show that happening live on a screen — without you standing up a Kafka/Flink cluster to do it.

Constraint	What it really means	Pass/Fail signal for judges
No heavy distributed pipelines	Don't use Kafka, Flink, Spark, or a hosted message queue. A single process (optionally with worker threads) must do the whole job.	You say "no Kafka" out loud during the pitch and show one process handling everything.
≥ 50,000 msgs/sec	Sustained throughput, not a burst. Measured over several seconds.	A live counter on the dashboard reads ≥50,000/sec continuously.
Millisecond alerts	Time from "message received" to "alert emitted" must be sub-millisecond to low-single-digit ms.	A live latency gauge reads well under 5ms (ideally <1ms).

Everything else (nice map, pretty UI) is secondary — it exists to *prove* the three rows above, not to impress on its own.

2. Verdict on the Proposed Solution (H3 + In-Memory Hash Set + Node Worker Threads + WebSockets)

Overall: the approach is correct and is the right one to build. It matches the constraint set almost exactly:

- Converting geofences to **H3 hexagon IDs at startup** and checking membership in a `Set` turns an $O(N)$ point-in-polygon problem into an **$O(1)$ string lookup**. This is the single most important design decision in the whole project — it's what makes 50k/sec possible on a laptop.
- Keeping everything **in RAM** (no DB writes per message) removes the actual bottleneck that kills "current systems" (disk I/O and PostGIS geometry queries).
- WebSockets** for pushing only violation events (not every message) keeps network load tiny.

Where the proposed plan is imprecise or risky, and how to fix it before you build:

1. **"50,000 eps" is ambiguous — decide *where* you measure it.** A single Node.js core doing `h3.latLngToCell()` + `set.has()` can process **well over 500,000–1,000,000 simple lookups/sec**. The lookup itself is never your bottleneck. The bottleneck is almost always *getting 50,000 messages/sec into the process* — serializing them, pushing them through a socket, or copying them across worker threads. So:
 - **Best approach for a 24h hackathon:** run the mock data generator *inside the same Node process* (or in a worker thread that pushes typed-array data, not JSON objects, via `SharedArrayBuffer`/transferable `ArrayBuffer`) rather than sending 50k JSON messages/sec over a real network socket. Sending 50k discrete JSON payloads/sec over TCP from a separate script is achievable but adds unnecessary risk and complexity you don't need to prove the concept.
 - If you *do* want a "real" network hop for demo credibility, use **UDP** (no handshake/ACK overhead) or a raw TCP stream with a compact binary frame format (fixed-width buffer: `[shipId:4][lat:4][lng:4][ts:8]` = 20 bytes/msg) instead of JSON. JSON parsing 50k times/sec is a measurable, avoidable cost.
2. **Worker Threads add overhead you may not need.** `postMessage` between threads uses structured cloning by default, which is *slow* for high-frequency small messages — it can become your bottleneck instead of solving one. If you use worker threads:
 - Use a `SharedArrayBuffer` + `Atomics` ring buffer to pass raw coordinate data between threads with **zero copying**, not `postMessage` with plain objects.
 - Or, simpler for a hackathon: **skip worker threads for the hot path entirely**. A single-threaded event loop doing H3 lookups can plausibly hit the throughput floor on its own, since the lookup is O(1) and cheap. Use a second thread only for the mock data generator (so generation doesn't starve the event loop), not for splitting the evaluation logic across cores. This is simpler to build, debug, and demo in 24 hours — and it's *more* impressive to say "one core, one thread, 50k/sec" than to hide behind parallelism.
3. **Don't emit a WebSocket message per processed packet, even for the metrics.** If you increment and broadcast a "processed count" on every single message, that itself becomes 50,000 WebSocket sends/sec and *will* choke your frontend and possibly your backend. Instead:
 - Increment a plain in-memory counter for every message (cheap).
 - On a `setInterval` every 200–250ms, compute `count / elapsed` and emit **one** metrics update to the frontend.
 - Only emit *individual* WebSocket messages for actual geofence violations (which should be rare relative to 50k/sec, keeping this cheap).
4. **H3 resolution choice matters and should be justified in your pitch.** H3 resolution ranges 0 (huge hexagons, ~1000km) to 15 (tiny, sub-meter). For a coastal EEZ/MPA use case, **resolution 6–8** is the sweet spot (hexagon edge ~3–36km) — fine enough to catch real boundary crossings, coarse enough that your forbidden-zone Set has a

manageable number of hexagons (hundreds to low thousands, not millions). Say this number out loud to judges — it shows you understand the design, not just copied a library call.

5. **The "50k/sec on standard hardware" claim needs a visible, repeatable proof, not a one-time spike.** Build the throughput counter to show a **rolling average over the last 1-2 seconds**, not an instantaneous peak, so a lucky burst can't be mistaken for sustained performance — and so you can't be caught out if it dips.

Bottom line: keep H3 + Hash Set + in-memory-only + sparse WebSocket alerts. Simplify the concurrency story to "single hot-path thread, separate generator thread, zero-copy data path" rather than a full worker-pool architecture — it's less risk for the same (or better) demoed throughput, and far more buildable in 24 hours.

3. What Dataset To Use

You need two different things, and conflating them is a common hackathon mistake:

A. A *realistic* dataset for authenticity (used at normal, low speed, for the "look, real ship tracks" part of the demo)

Real, free, no-signup-required AIS datasets:

- **Danish Maritime Authority (DMA) AIS data** — free daily CSV/ZIP downloads of real AIS positions (MMSI, lat, lon, timestamp, speed over ground, course over ground) covering Danish waters. Extremely popular for prototyping because it's clean and immediately usable.
- **NOAA Marine Cadastre AIS data (U.S. waters)** — large, free, historical AIS datasets by year/zone, very well documented, good coordinate coverage for U.S. coastlines.
- **Kaggle "Vessel/Ship AIS Traffic" datasets** — smaller, pre-cleaned CSVs, faster to get running if you don't want to deal with the raw government ZIP files.

Use one of these to seed **realistic starting coordinates, headings, and speeds** for your simulated fleet, and to draw a genuinely real coastline/EEZ boundary for your geofence (rather than a made-up polygon) — this makes the demo feel grounded rather than arbitrary.

B. A *synthetic, high-frequency* generator for the throughput/stress-test part of the demo

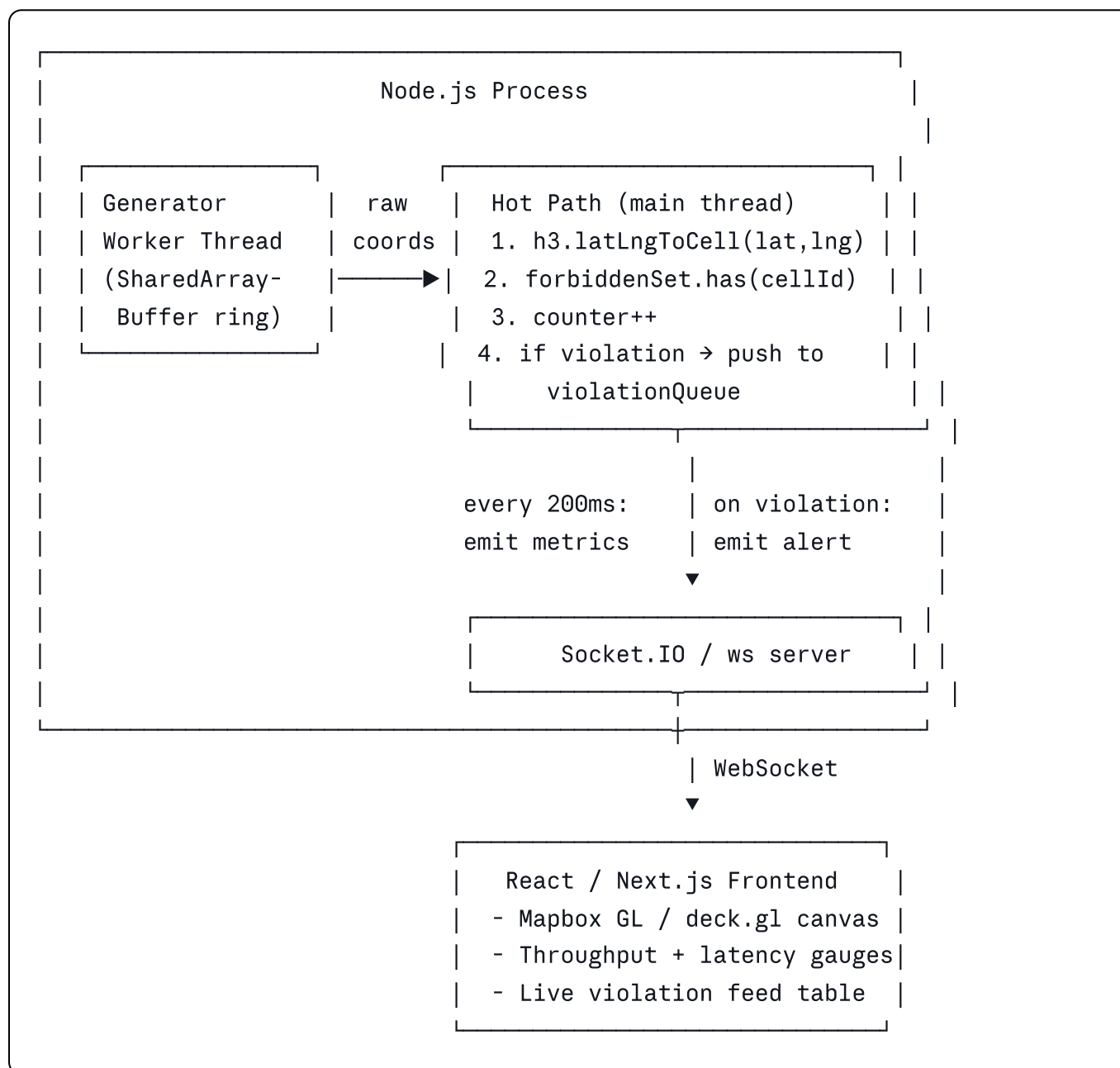
No real-world AIS feed actually arrives at 50,000 msgs/sec for a small region — real AIS is bursty and much lower volume. To hit and *prove* the throughput floor, you must synthetically generate messages:

- Take ~500–2,000 "virtual vessels" seeded from the real dataset's starting positions.
- On each tick, update each vessel's lat/lon slightly (small random walk or straight-line movement along a heading) and re-emit its position.

- Multiply the emission rate (or the number of virtual vessels ticking per interval) until your counter shows $\geq 50,000/\text{sec}$ sustained.
- Deliberately route a controlled fraction of vessels through your forbidden hexagons on a timer, so violations appear on camera at predictable moments during the live demo (don't leave this to chance).

Recommendation: Build the synthetic generator first (it's what proves the throughput constraint and is required regardless), and layer in the real dataset only for visual/narrative polish once the core engine works. If time runs short, the synthetic-only version is still a fully valid, constraint-satisfying submission.

4. Final Architecture



Backend

- **Runtime:** Node.js (TypeScript recommended if your team is comfortable — catches bugs faster under time pressure; plain JS is fine too).

- **Spatial index:** `h3-js` (Uber H3). At startup: load your geofence polygon(s) → `h3.polygonToCells(polygon, resolution)` → store resulting cell IDs in a native `Set`.
- **Hot loop:** for every incoming coordinate → `h3.latLngToCell(lat, lng, resolution)` → `forbiddenSet.has(cellId)`.
- **Generator:** separate worker thread (or even a separate lightweight loop using `setImmediate`/a tight `for` loop with batching) writing into a `SharedArrayBuffer` ring buffer that the main thread drains — avoids `postMessage` cloning overhead.
- **Metrics:** plain counters (`messagesProcessed`, `violationsDetected`), sampled and emitted every 200ms via `setInterval`.
- **Alert transport:** `socket.io` or the plain `ws` library. Emit alert payloads only (vessel ID, lat, lng, hex ID, timestamp) — tiny JSON, low frequency.

Frontend

- **Framework:** React (Next.js is fine, plain Vite+React is faster to scaffold in a hackathon if you don't need SSR).
- **Map rendering: do not use individual DOM markers.** Use `Mapbox GL JS` with a GL layer, or `deck.gl`'s `ScatterplotLayer`/`HexagonLayer` for GPU-accelerated rendering of thousands of points. If you want zero external API keys, a plain `<canvas>` element you draw directly to (via `requestAnimationFrame`) is a completely acceptable, dependency-free fallback.
- **Metrics panel:** two large, live-updating numbers — "Messages/sec" and "Latency (ms)" — plus a small rolling line chart (e.g. `recharts` or `chart.js`) of throughput over the last 30–60 seconds. This is the single highest-value UI element for judging.
- **Violation feed:** a scrolling table: `Vessel ID | Time | Lat, Lng | Hex ID`.

Key libraries

Purpose	Library
Spatial indexing	<code>h3-js</code>
Real-time transport	<code>socket.io</code> (or <code>ws</code>)
Map/canvas rendering	<code>mapbox-gl</code> or <code>deck.gl</code> , or raw <code><canvas></code>
Charts	<code>recharts</code> or <code>chart.js</code>
CSV parsing (real dataset)	<code>papaparse</code>

5. Hour-by-Hour 24-Hour Plan

Hours	Task
-------	------

0-1	Scaffold repo: Node backend + Vite/React frontend. Install <code>h3-js</code> , <code>socket.io</code> , <code>mapbox-gl</code> or set up canvas.
1-3	Download and inspect real AIS sample (DMA or NOAA). Extract a small bounding box + a handful of real vessel start points. Define one real forbidden polygon (e.g. a marine protected area) using real coordinates.
3-5	Backend: startup routine — <code>polygonToCells()</code> → build <code>forbiddenSet</code> . Write and unit-test the single-message evaluator function in isolation (feed it known in-zone / out-of-zone coordinates, confirm correctness).
5-8	Build the synthetic generator (worker thread + SharedArrayBuffer ring, or simple batched loop). Get it emitting into the hot path. Add the <code>messagesProcessed</code> counter. Verify raw throughput via console logging before touching the network layer at all.
8-10	Wire up <code>socket.io</code> : metrics broadcast every 200ms, violation broadcast on detection. Confirm counter climbs past 50,000/sec in the raw engine.
10-13	Frontend: metrics panel (throughput number, latency number, rolling chart) wired to WebSocket. Get this working <i>before</i> the map — it's your strongest proof point.
13-17	Frontend: map layer (Mapbox/deck.gl/canvas). Render the forbidden zone polygon. Render a sampled subset of live vessel positions (don't try to render literally 50,000 DOM/GL points if it's not needed — sampling for visual display is fine, since the <i>engine</i> is what's being judged on volume, not the pixel count).
17-19	Violation feed table + visual pulse/flash on the map when a violation event arrives.
19-21	Deliberately script 2-3 vessels to cross into the forbidden zone at predictable times so the live demo reliably shows a violation on camera.
21-23	Stress test: run it for several continuous minutes, confirm the throughput and latency numbers stay stable, not just at t=0. Fix any memory growth (make sure you aren't accidentally retaining every processed message somewhere).
23-24	Polish: rehearse the 2-minute pitch (see §7), fix visual rough edges, prepare fallback (recorded video/GIF) in case live demo Wi-Fi or projector fails.

6. Problems You're Likely To Hit, and Fixes

Problem	Cause	Fix
Throughput counter plateaus well under 50,000/sec	Generator itself can't produce messages fast enough, or <code>postMessage</code> cloning between threads is the bottleneck	Batch messages (send arrays, not one call per message), or switch to <code>SharedArrayBuffer</code> /typed arrays instead of object cloning
Frontend freezes or lags	Rendering every single point as a DOM element, or re-rendering React state on every message	Use canvas/WebGL layer; throttle React state updates to the same 200ms cadence as your metrics, not per-message
Latency numbers look suspiciously perfect (0.00ms)	Clock resolution too coarse (<code>Date.now()</code>), or you're timing the wrong span	Use <code>process.hrtime.bigint()</code> for nanosecond-resolution timing around just the lookup step
Memory grows over time / eventual crash	Accumulating every processed (non-violating) message in an array instead of dropping it	Explicitly discard — don't push non-violations anywhere; only track counters
Hexagon set is huge and lookups feel slow to explain	H3 resolution set too fine (e.g. resolution 10+) for a large polygon, producing millions of cells	Use resolution 6–8 for a coastal-scale zone; explain the trade-off in your pitch
"Real-time" demo shows no violations because random walk never crosses the zone	Left it to chance	Script specific vessels with deterministic paths that cross the boundary at known times
Judges ask "why not Kafka" and team has no answer	Didn't internalize <i>why</i> the constraint exists	Rehearse the one-line answer: "Kafka/Flink add network + disk + serialization overhead we don't need — H3 turns geofencing into an O(1) in-memory lookup, so a single process can already clear the throughput floor without a distributed system."

7. The Pitch (say this, roughly, in the demo)

1. "Traditional maritime tracking writes every AIS ping to a spatial database and runs point-in-polygon geometry per message — that bottlenecks long before 50,000 messages a second."
2. "We instead pre-tile our protected zones into H3 hexagons at startup, store them in an in-memory hash set, and turn every incoming ship position into a single $O(1)$ lookup."
3. *(point at metrics panel)* "You're watching us process [X],000+ messages a second right now, with sub-millisecond evaluation latency, on a single laptop — no Kafka, no Flink, no database in the hot path."
4. *(point at map as a scripted vessel crosses the boundary)* "The moment a vessel's hexagon matches a forbidden cell, we push a single WebSocket alert — that's the flash you just saw, end-to-end in under a millisecond."

This document is intentionally implementation-ready: the architecture, library choices, resolution numbers, and hour plan above are the actual decisions to start coding against — no further design discussion should be needed before you open your editor.